

from Beginner to Master

# DevOps를 위한 **Docker** 가상화

Section 0. Cloud Native Technologies

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명



Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

수정하기

이 되었던 때가 있었습니  
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구  
사람입니다. 개발자로서, 강  
지만, 그래도, 남들보다 조

ecture, Blockchain,

- **Section 0: Introduction to Cloud Native Technologies**
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- Section 3: Docker Network and Storage
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 0.

# Cloud Native Technologies

- 소프트웨어 아키텍처
- 컨테이너 가상화 기술과 Docker





## 소프트웨어 아키텍처란?

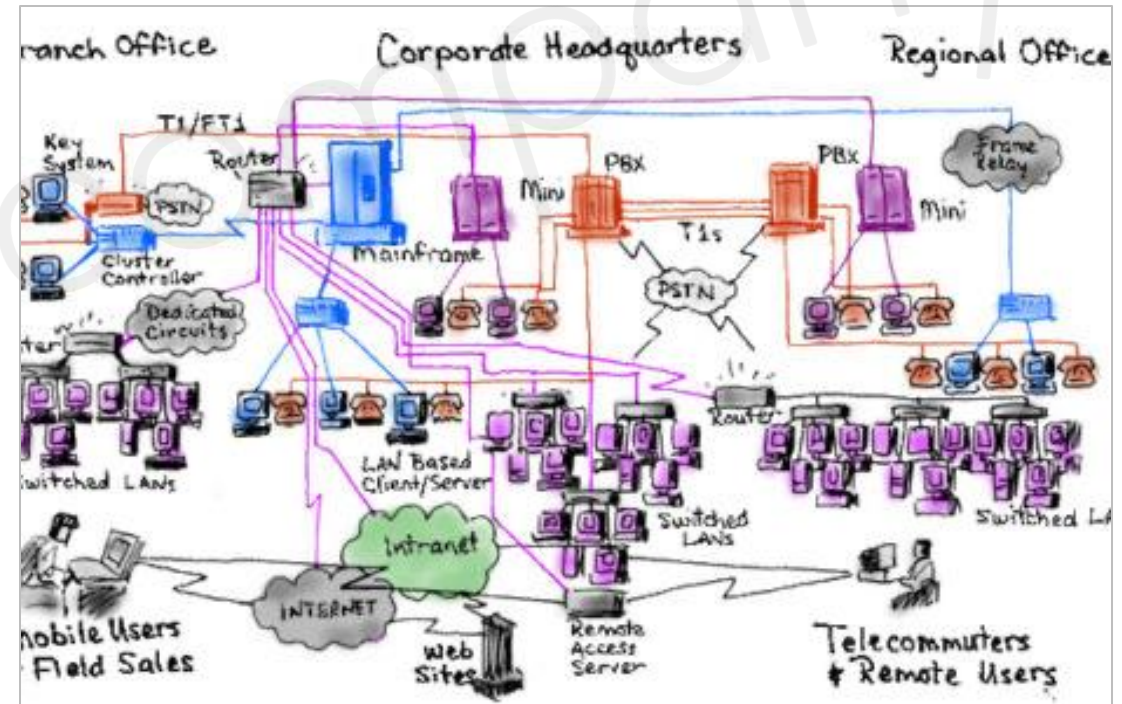
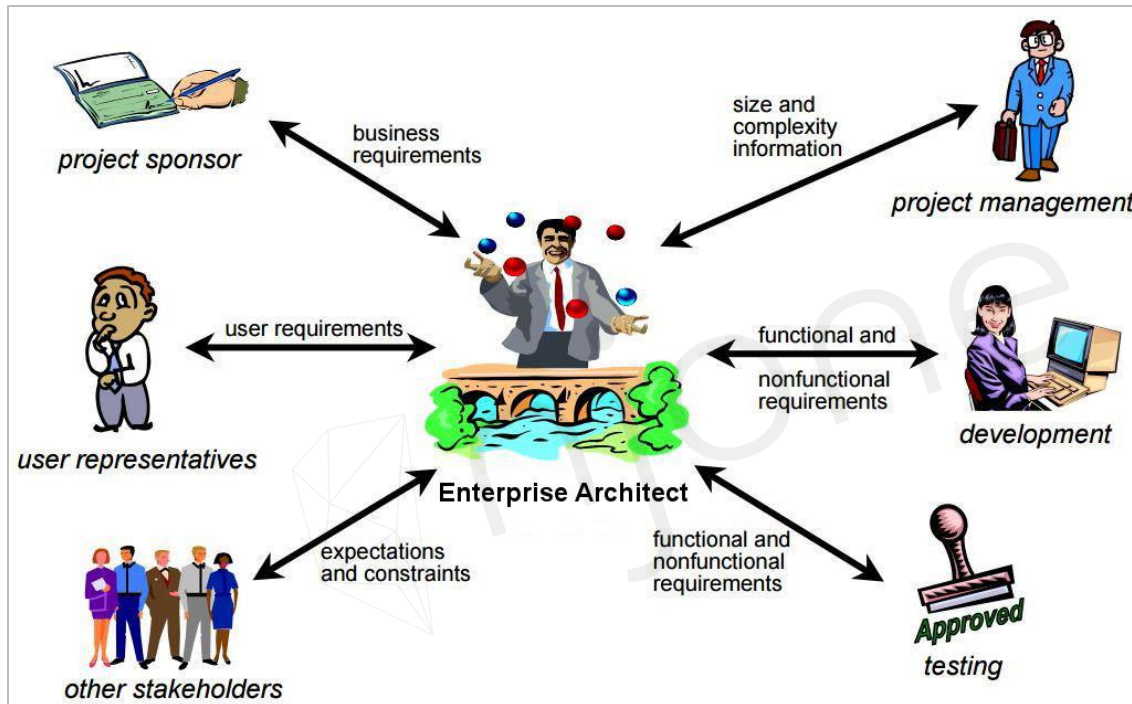
"Architecture is both the *process* and the *product* of *planning*, *designing*, and *constructing* buildings or any other structures. Architectural works, in the material form of buildings, are often perceived as cultural symbols and as works of art. Historical civilizations are often identified with their surviving architectural achievements. "



## 소프트웨어 아키텍처란?

- 무형의 지식 집합체(코드)
- 기능적인 측면 보다는 안정적인 운영을 위한 전체적인 구조를 설계

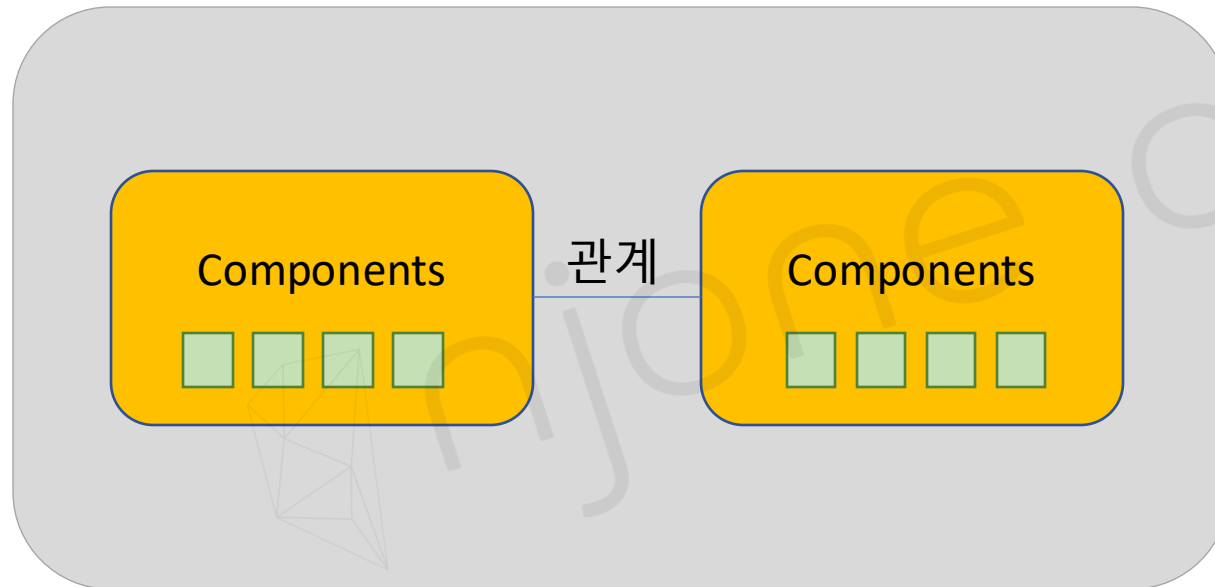
ex) 오케스트라



## 소프트웨어 아키텍처란?

- SW를 구성하는 요소와 요소 간의 관계 정의
  - | 전체 구성 관계, 포함 관계, 호출 관계
  - | SW 설계자, 개발, 사용자 등 이해 관계자들 간의 커뮤니케이션 도구

소프트웨어

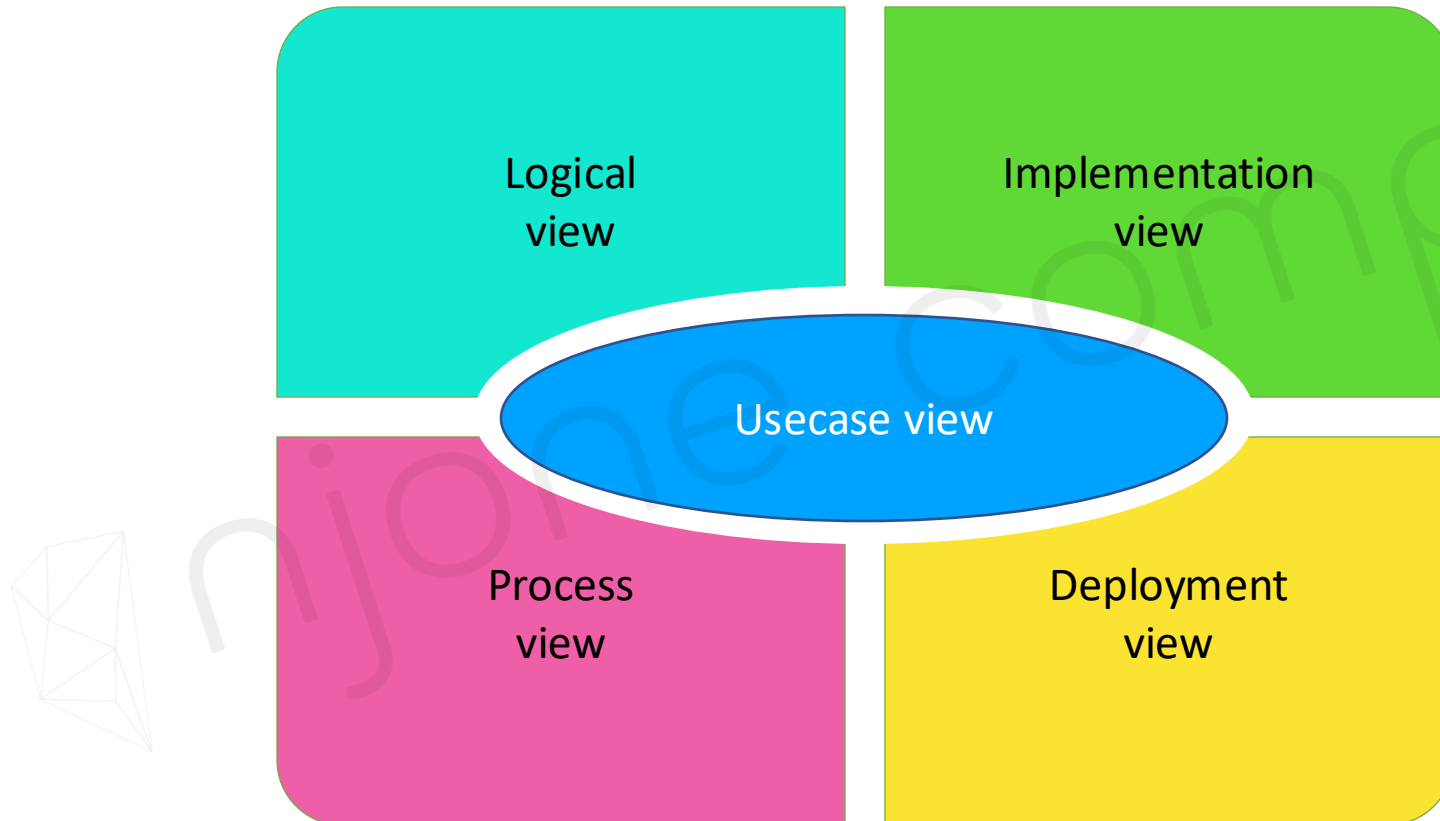




## 소프트웨어 아키텍처란?

- 이해관계자들이 시스템을 이해하는 수준은 모두 다름 → 관점 → View

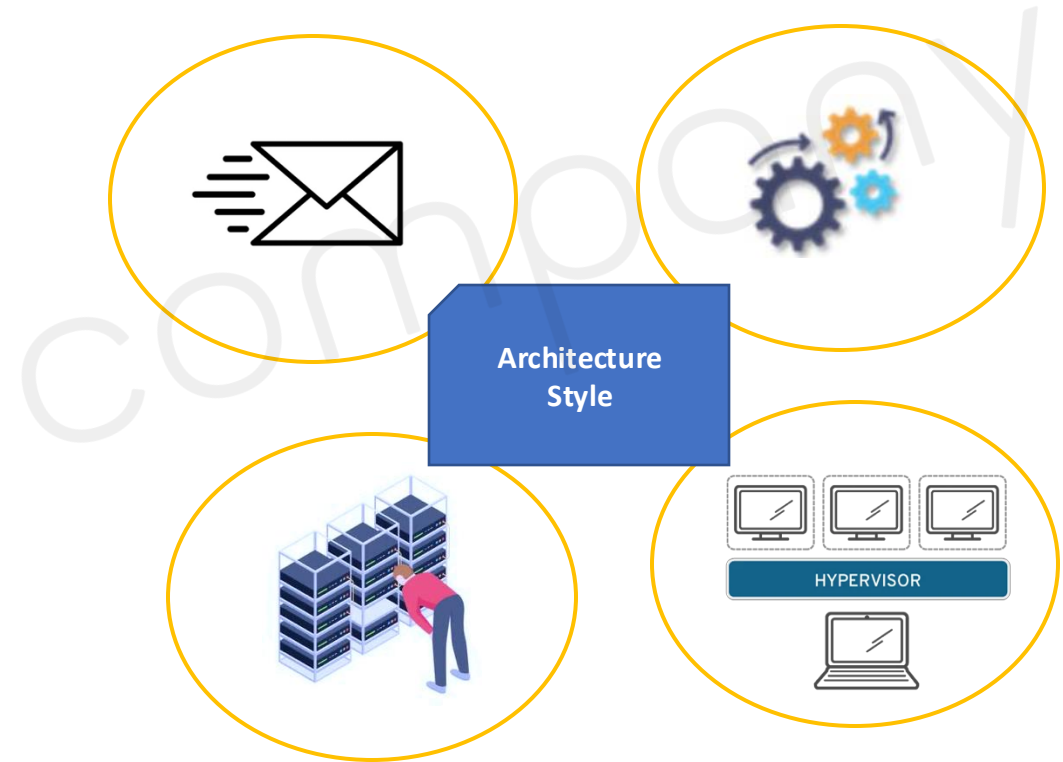
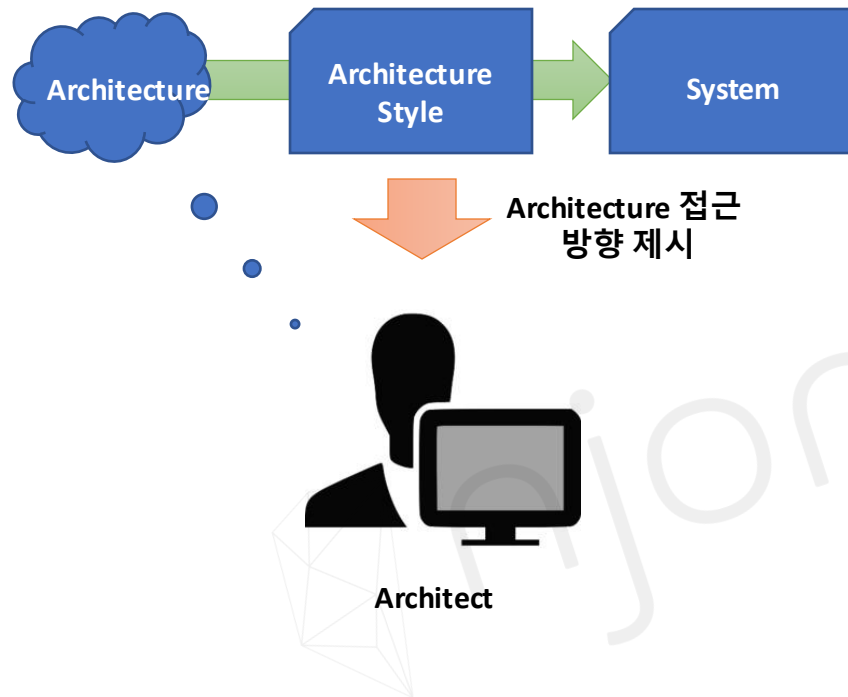
| UML(Unified Modeling Language)





## 소프트웨어 아키텍처란?

- Architecture 스타일과 패턴



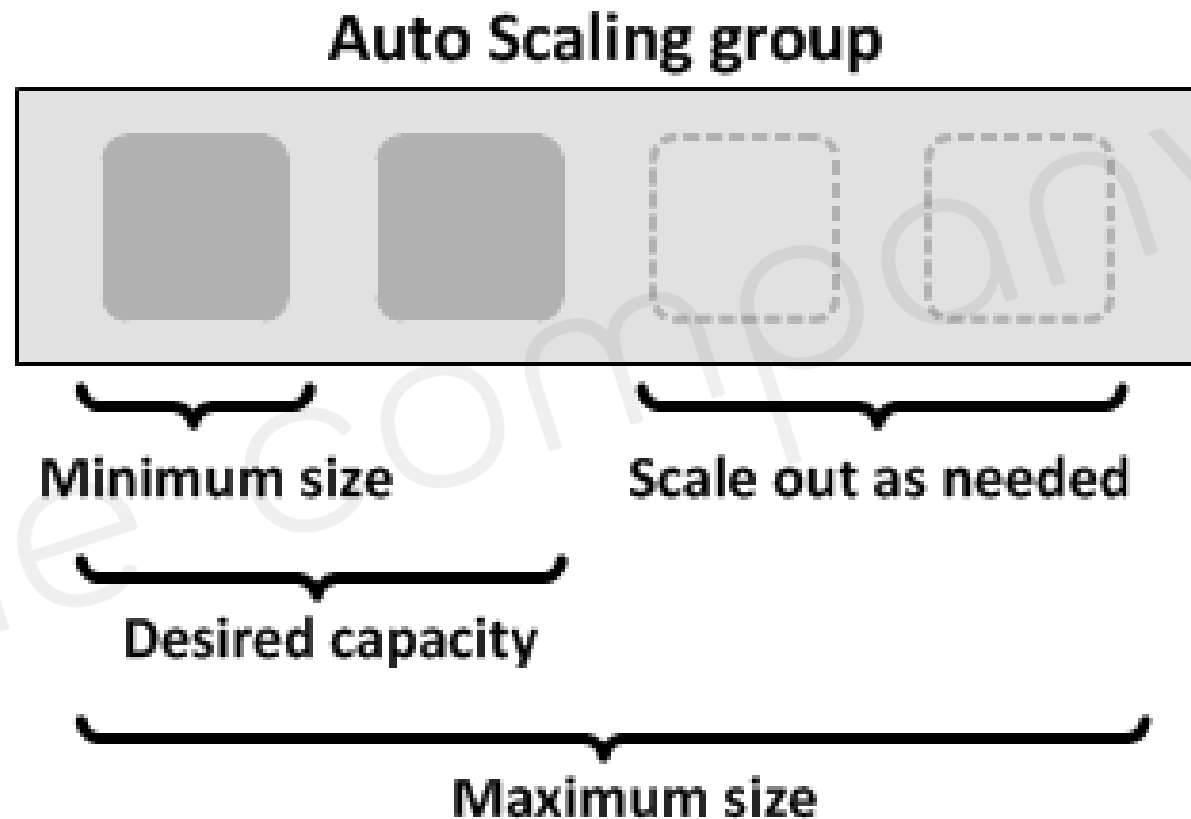
## IT 시스템의 역사

- 1960 ~ 1980s : Fragile, Cowboys
  - | Mainframe, Hardware
- 1990 ~ 2000s : Robust, Distributed
  - | Changes
- 2010s ~ : Resilient, **Anti-Fragile, Cloud Native**
  - | Flow of value의 지속적인 개선

| Domain              | Fragile        | Robust      | Anti-Fragile        |
|---------------------|----------------|-------------|---------------------|
| Transport           | Racing car     | Tank, 4x4   | Horse               |
| Transport in London | Train          | Bus         | Bicycle             |
| Market              | Stock exchange | Supermarket | Bazaar              |
| Knowledge           | Science book   | Journal     | Wiki                |
| IT Culture          | Cowboys        | ITIL        | <b>DevOps</b>       |
| IT Architecture     | Monolithic     | Distributed | <b>Cloud Native</b> |

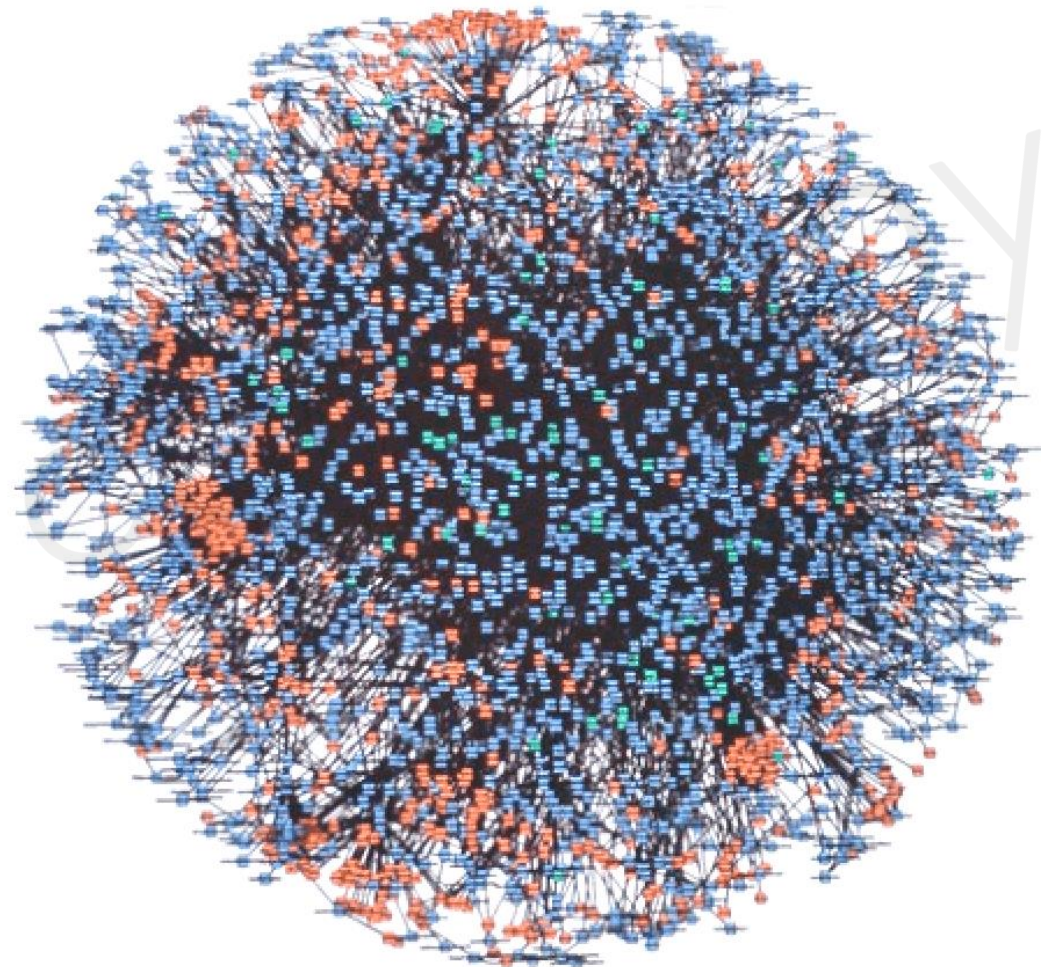
## Antifragile

- Auto scaling
- Microservices
- Chaos engineering
- Continuous deployments



## Antifragile

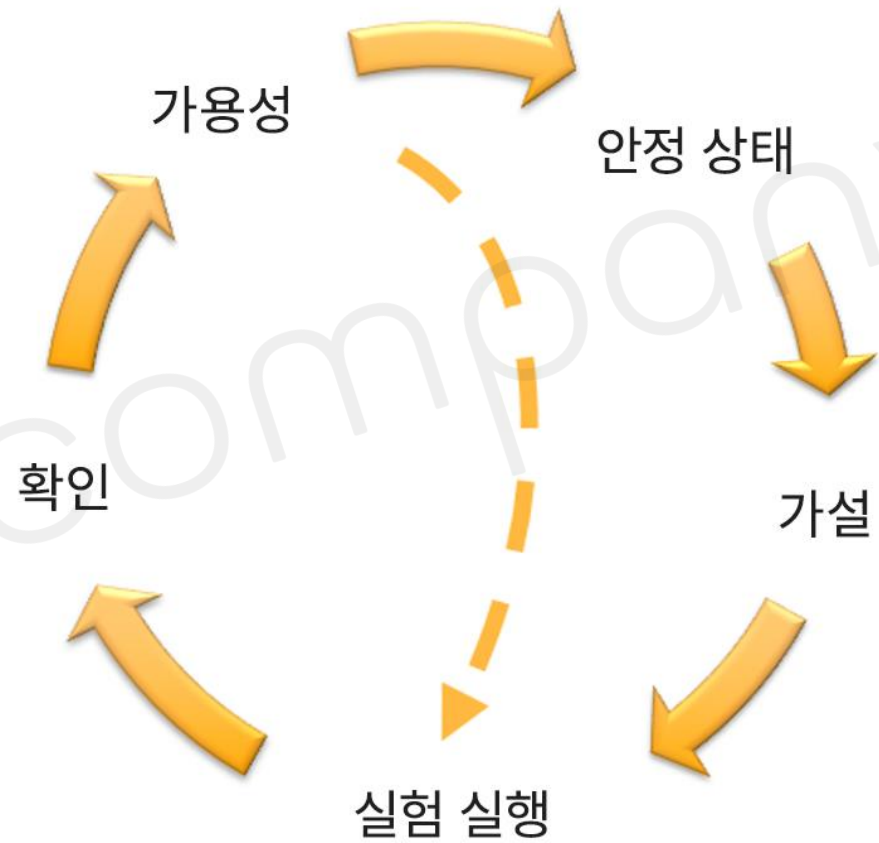
- Auto scaling
- **Microservices**
- Chaos engineering
- Continuous deployments



## Antifragile

- Auto scaling
- Microservices
- Chaos engineering
- Continuous deployments

- 변동
- 예견된 불확실성
- 예견되지 않는 불확실성
- 카오스 불확실성
- 의도적인 시스템 장애





## Antifragile

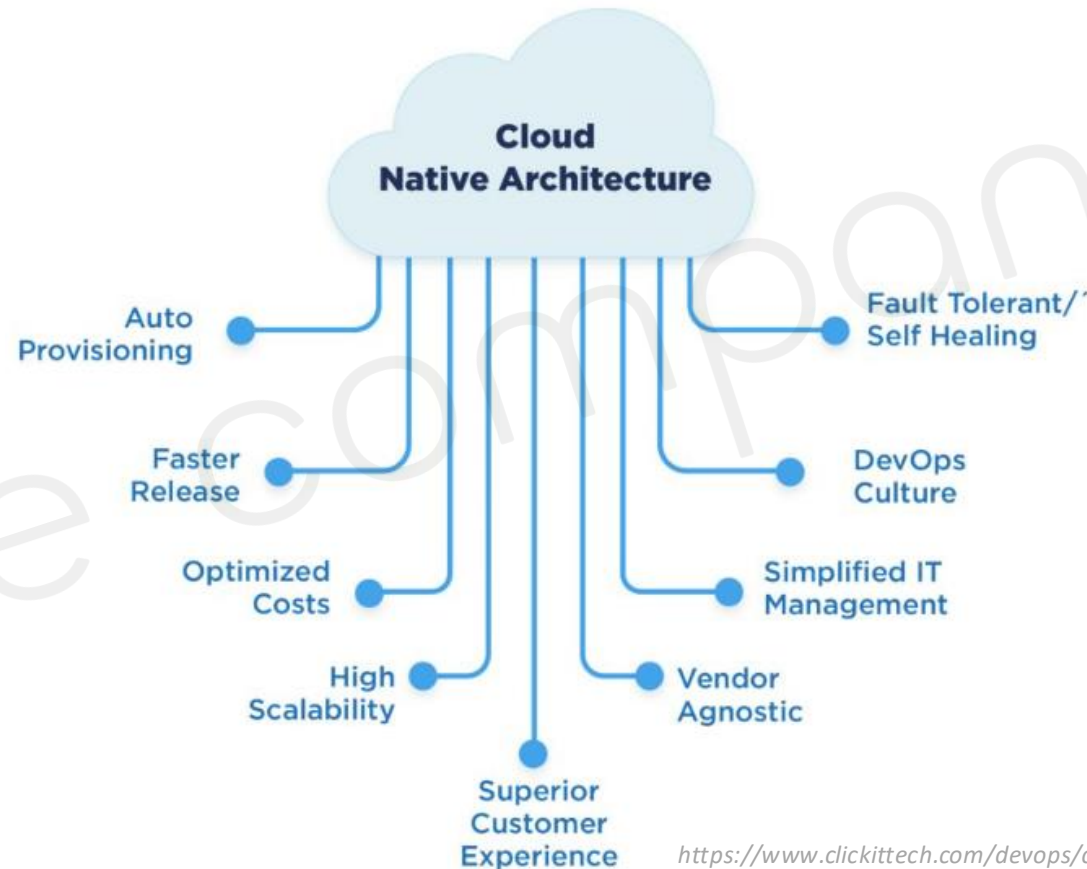
- Auto scaling
- Microservices
- Chaos engineering
- Continuous deployments



## Cloud Native Architecture의 소개

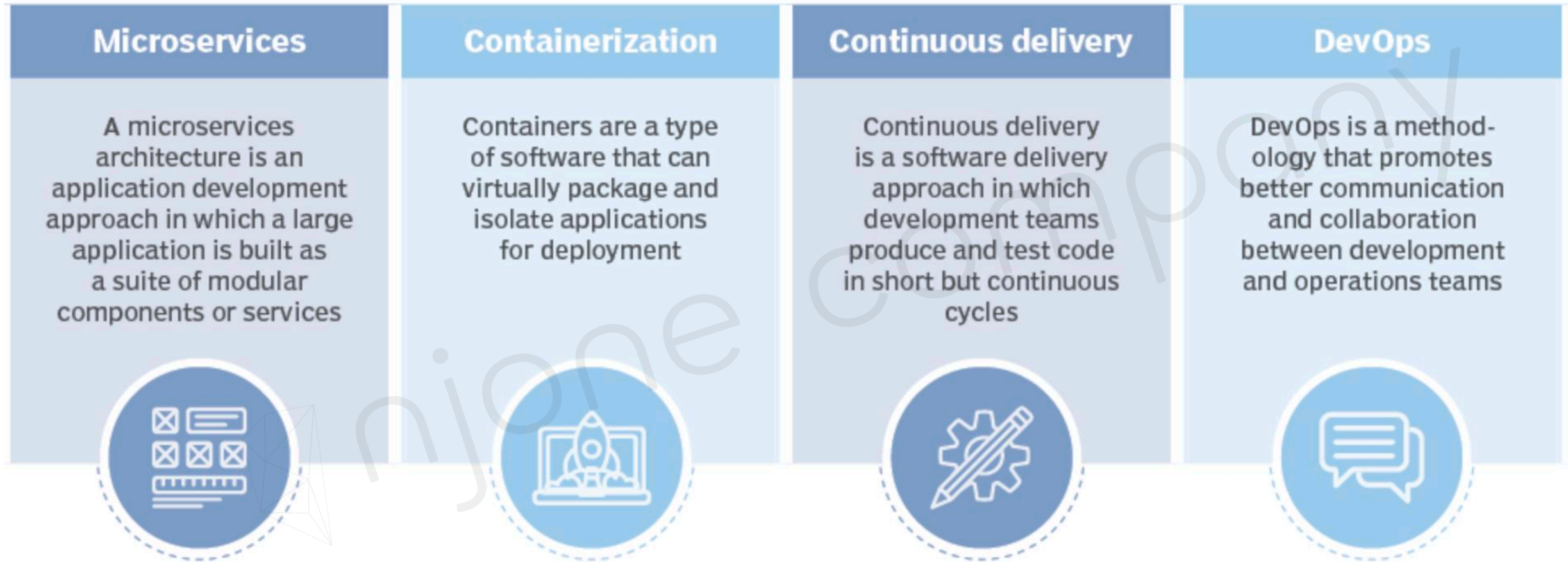
- 확장 가능한 아키텍처
- 탄력적 아키텍처
- 장애 격리
- MSA Application
- DevOps and CI/CD
- Cost Efficiency
- Innovative Technologies

Benefits of a Cloud Native Architecture



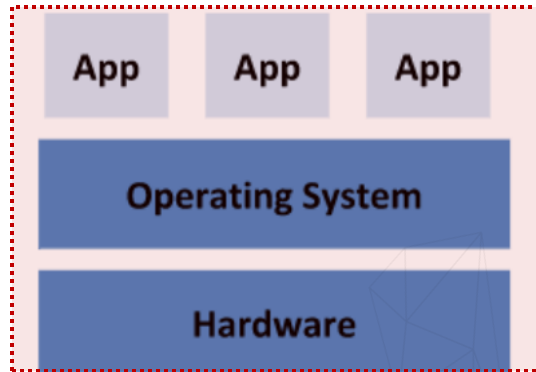
## Cloud Native Application 구성 컴포넌트

- <https://www.cncf.io/blog/2018/03/08/introducing-the-cloud-native-landscape-2-0-interactive-edition/>



## 가상화 기술의 이해

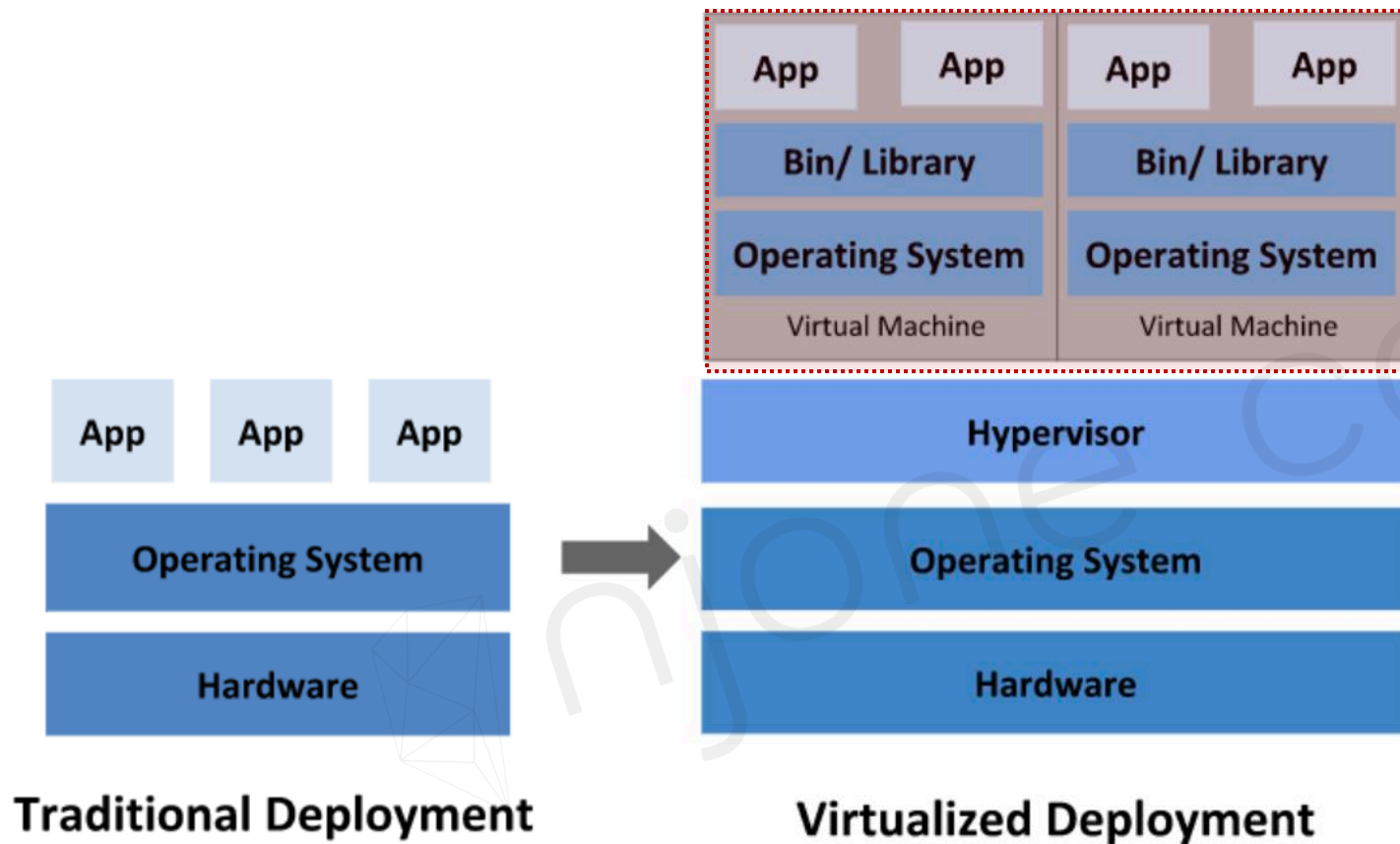
- 기존 가상화 방식



**Traditional Deployment**

## 가상화 기술의 이해

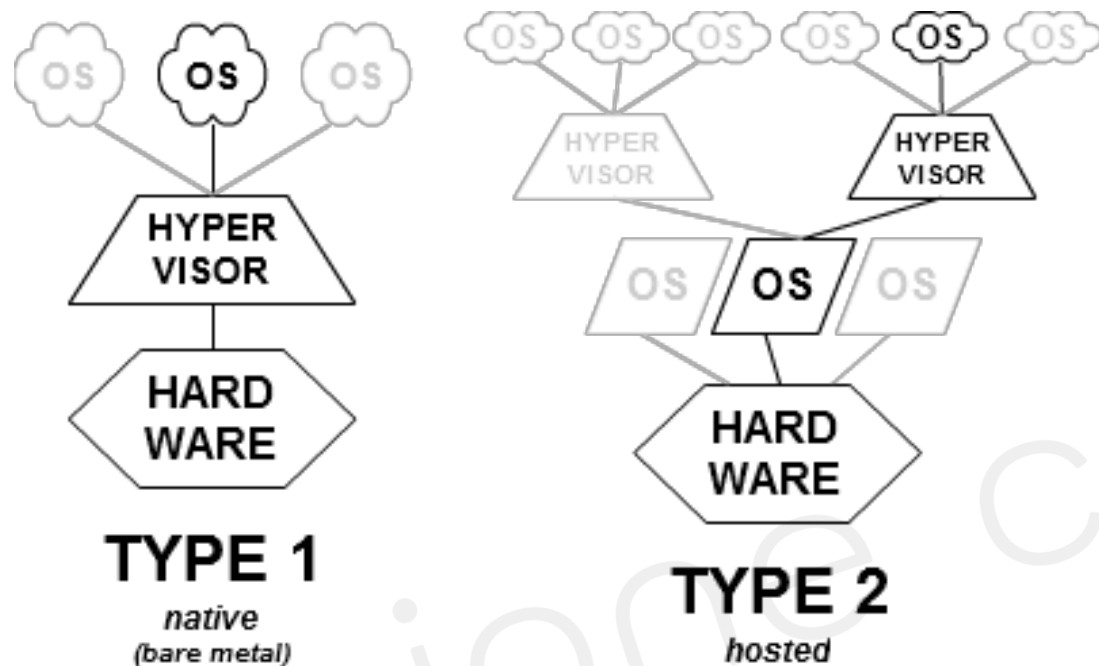
- 기존 가상화 방식 → 서버 가상화





# 컨테이너 가상화 기술과 Docker

## 가상화 기술의 이해



- 거대한 이미지 사이즈
- 느린 시작 시간
- VM 간의 환경 불일치

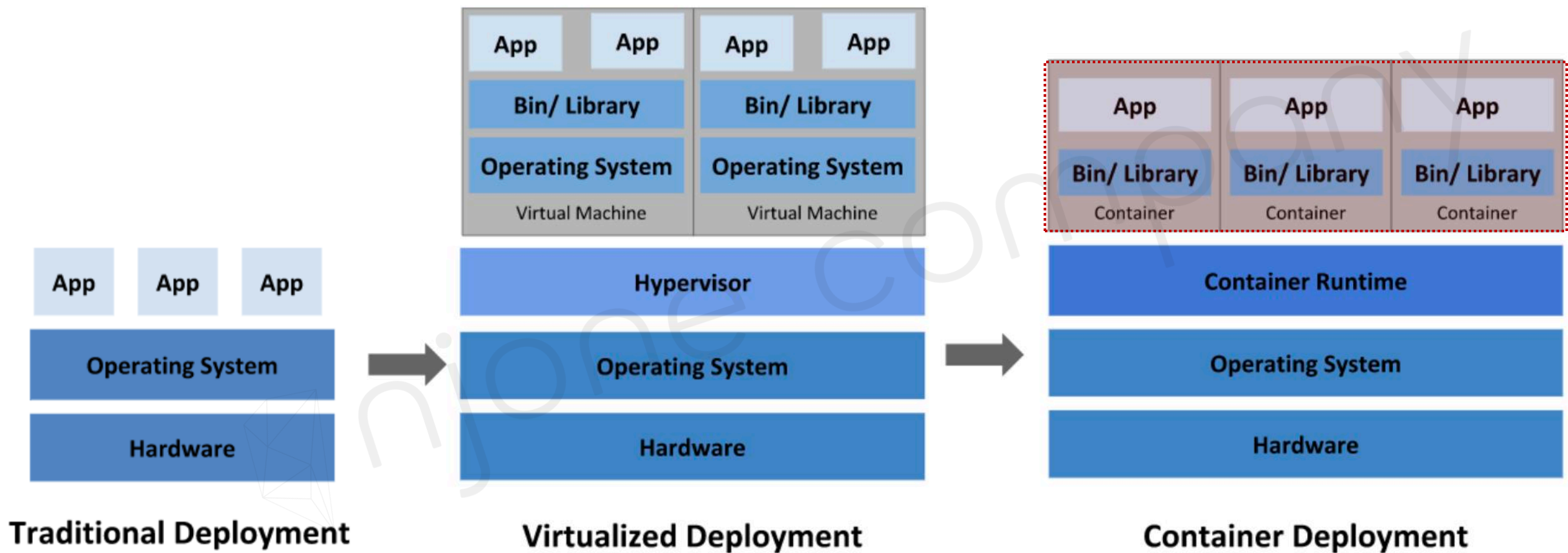
하이퍼바이저 타입 (<https://ko.wikipedia.org/wiki/하이퍼바이저>)

Traditional Deployment

Virtualized Deployment

## 가상화 기술의 이해

- 기존 가상화 방식 → 서버 가상화 → 컨테이너 가상화



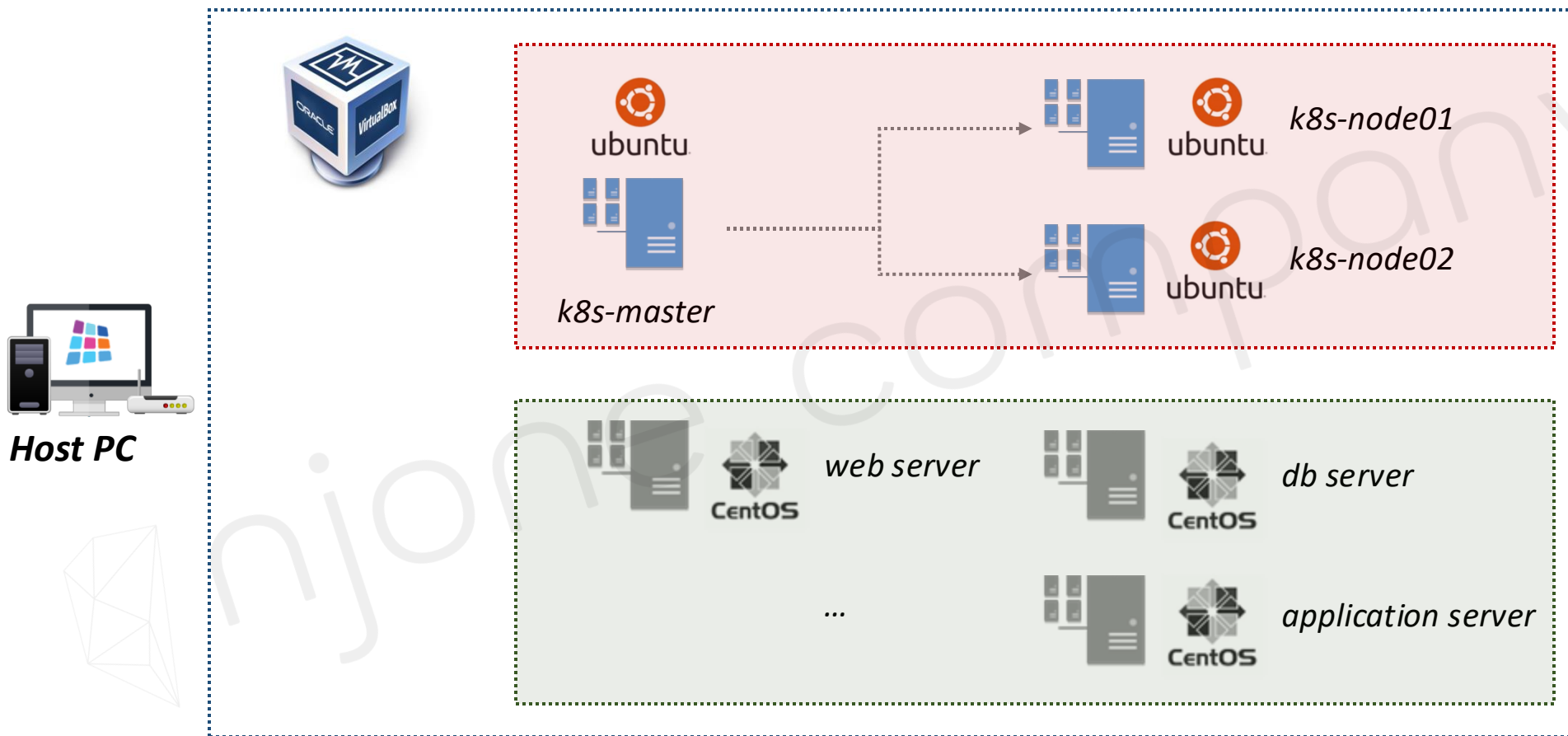
## 가상화 기술의 이해

- 기존 가상화 방식 → 서버 가상화 → 컨테이너 가상화

|                | 하이퍼 바이저 가상화                                  | 컨테이너 가상화                                        |
|----------------|----------------------------------------------|-------------------------------------------------|
| 시작 시간          | 몇 분                                          | 몇 초                                             |
| 이미지 크기         | 수 GB ~ 수백 GB<br>(OS + Application + Runtime) | ~ 수백 MB<br>(Application + Runtime)              |
| 이식성            | 대부분 가상 이미지에 대한 변환이 필요                        | 컨테이너 이미지 그대로 사용 가능                              |
| Guest OS       | Windows / Linux                              |                                                 |
| Guest OS와 의 관계 | Guest OS를 별도의 가상 하드웨어로 인식                    | Host OS를 커널 수준으로 분리 가능<br>필요 시 Host와 리소스 공유 가능  |
| 네트워크           | Bridge, Host, Container                      |                                                 |
| 데이터 관리         | VM 내부 또는 연결 된 스토리지에 저장                       | 컨테이너 종료 시 소멸되는 컨테이너 내부 스토리지와 컨테이너 외부 스토리지 사용 가능 |

## 서버 가상화 예

- VirtualBox + Linux (Ubuntu / CentOS)



## 컨테이너 가상화 예

- Docker Engine

